

1. (7 pts): Weiss, Exercise 7.11 (show the intermediate steps as an array not a tree)

Show how heapsort processes the input: | 142 543 | 123 | 65 453 | 879 572 | 434 | 111 | 242 811 | 102 |

Build Heap

	POSITION	0	1	2	3	4	5	6	7	8	9	10	11	12
initial arr	EMPTY	142,	543,	123,	65,	453,	879,	572,	434,	111,	242,	811,	102	
percolateDn	EMPTY	142,	543,	123,	65,	811,	879,	572,	434,	111,	242,	453,	102	
percolateDn	EMPTY	142,	543,	123,	434,	811,	879,	572,	65,	111,	242,	453,	102	
percolateDn	EMPTY	142,	543,	879,	434,	811,	123,	572,	65,	111,	242,	453,	102	
percolateDn	EMPTY	142,	811,	879,	434,	543,	123,	572,	65,	111,	242,	453,	102	
percolateDn	EMPTY	879,	811,	572,	434,	543,	123,	142,	65,	111,	242,	453,	102	

Heap Sort

	POSITION	0	1	2	3	4	5	6	7	8	9	10	11	12
initial arr	EMPTY	879,	811,	572,	434,	543,	123,	142,	65,	111,	242,	453,	102	
deleteMax	EMPTY	102,	811,	572,	434,	543,	123,	142,	65,	111,	242,	453,	879*	
percolateDn	EMPTY	811,	102,	572,	434,	543,	123,	142,	65,	111,	242,	453,	879*	
percolateDn	EMPTY	811,	543,	572,	434,	102,	123,	142,	65,	111,	242,	453,	879*	
percolateDn	EMPTY	811,	543,	572,	434,	242,	123,	142,	65,	111,	102,	453,	879*	
deleteMax	EMPTY	453,	543,	572,	434,	242,	123,	142,	65,	111,	102,	811*,	879*	
percolateDn	EMPTY	572,	543,	453,	434,	242,	123,	142,	65,	111,	102,	811*,	879*	
deleteMax	EMPTY	102,	543,	453,	434,	242,	123,	142,	65,	111,	572*,	811*,	879*	
percolateDn	EMPTY	543,	102,	453,	434,	242,	123,	142,	65,	111,	572*,	811*,	879*	
percolateDn	EMPTY	543,	434,	453,	102,	242,	123,	142,	65,	111,	572*,	811*,	879*	
percolateDn	EMPTY	543,	434,	453,	111,	242,	123,	142,	65,	102,	572*,	811*,	879*	
deleteMax	EMPTY	102,	434,	453,	111,	242,	123,	142,	65,	543*,	572*,	811*,	879*	
percolateDn	EMPTY	453,	434,	102,	111,	242,	123,	142,	65,	543*,	572*,	811*,	879*	
percolateDn	EMPTY	453,	434,	123,	111,	242,	102,	142,	65,	543*,	572*,	811*,	879*	
deleteMax	EMPTY	65,	434,	123,	111,	242,	102,	142,	453*,	543*,	572*,	811*,	879*	
percolateDn	EMPTY	434,	65,	123,	111,	242,	102,	142,	453*,	543*,	572*,	811*,	879*	
percolateDn	EMPTY	434,	242,	123,	111,	65,	102,	142,	453*,	543*,	572*,	811*,	879*	
deleteMax	EMPTY	142,	242,	123,	111,	65,	102,	434*,	453*,	543*,	572*,	811*,	879*	
percolateDn	EMPTY	242,	142,	123,	111,	65,	102,	434*,	453*,	543*,	572*,	811*,	879*	
deleteMax	EMPTY	102,	142,	123,	111,	65,	242*,	434*,	453*,	543*,	572*,	811*,	879*	
percolateDn	EMPTY	142,	102,	123,	111,	65,	242*,	434*,	453*,	543*,	572*,	811*,	879*	
percolateDn	EMPTY	142,	111,	123,	102,	65,	242*,	434*,	453*,	543*,	572*,	811*,	879*	
deleteMax	EMPTY	65,	111,	123,	102,	142*,	242*,	434*,	453*,	543*,	572*,	811*,	879*	
percolateDn	EMPTY	123,	111,	65,	102,	142*,	242*,	434*,	453*,	543*,	572*,	811*,	879*	
deleteMax	EMPTY	102,	111,	65,	123*,	142*,	242*,	434*,	453*,	543*,	572*,	811*,	879*	
percolateDn	EMPTY	111,	102,	65,	123*,	142*,	242*,	434*,	453*,	543*,	572*,	811*,	879*	
deleteMax	EMPTY	65,	102,	111*,	123*,	142*,	242*,	434*,	453*,	543*,	572*,	811*,	879*	
percolateDn	EMPTY	102,	65,	111*,	123*,	142*,	242*,	434*,	453*,	543*,	572*,	811*,	879*	
deleteMax	EMPTY	65*,	102*,	111*,	123*,	142*,	242*,	434*,	453*,	543*,	572*,	811*,	879*	

2. (7 pts): Weiss, Exercise 7.19

Sort 3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5 using quicksort with median-of-three partitioning and a cutoff of 3.

QuickSort

$$left = 3$$
$$right = 5$$
$$center = 9$$

pivot = 5

move in until $i \geq pivot$

move in until $j \leq pivot$

```

                                P
pivot [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
        i                                j

```

```
move  [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
           i           j
```

```
swap  [3, 1, 4, 1, 3, 9, 2, 6, 5, 5, 5]
           i                j
```

```
move  [3, 1, 4, 1, 3, 9, 2, 6, 5, 5, 5]
           i  j
```

```
swap  [3, 1, 4, 1, 3, 2, 9, 6, 5, 5, 5]
      i  j
```

```
move  [3, 1, 4, 1, 3, 2, 9, 6, 5, 5, 5]
           j  i
```

```
swap [3, 1, 4, 1, 3, 2, 5, 6, 5, 5, 9]
pivot i
```

P
P
 pivot [3, 1, 4, 1, 3, 2] [5, 6, 9, 5, 5]
i
j
i
j

```
move  [3, 1, 4, 1, 3, 2] [5, 6, 9, 5, 5]
      i          j      i          j
```

```
swap  [1, 1, 4, 3, 3, 2] [5, 5, 9, 6, 5]
      i          j      i          j
```

```
move  [1, 1, 4, 3, 3, 2] [5, 5, 9, 6, 5]
      j  i              j  i
```

```
swap  [1, 1, 2, 3, 3, 4] [5, 5, 5, 6, 9]
pivot      i              i
```

FINAL [1, 1, 2, 3, 3, 4, 5, 5, 5, 6, 9]

3. (7 pts): Weiss, Exercise 7.24

Construct a permutation of 20 elements that is as bad as possible for quicksort using median-of-three partitioning and a cutoff of 3.

The following set of numbers will cause the algorithm to be $O(n^2)$:

9,2,1,3,4,5,6,0,7,8,19,12,11,13,14,15,16,10,17,18

4. (7 pts): Weiss Exercise 9.1

Find a topological ordering for the graph in Figure 9.81.

Order: s,G,D,A,B,H,E,I,F,C,t

5. (7 pts): Weiss Exercise 9.7a

a. Give an example where Dijkstra's algorithm gives the wrong answer in the presence of a negative edge but no negative-cost cycle.

An example of this could be a directed graph of 3 edges $A \rightarrow B \rightarrow C \mid A \rightarrow C$:

The edge weights for this graph are as follows:

$A \rightarrow B = 6$

$B \rightarrow C = -5$

$A \rightarrow C = 3$

Starting from A, we get the following

v	known	dv	pv
A	F	0	0
B	F	inf	0
C	F	inf	0

v	known	dv	pv
A	T	0	0
B	F	6	A
C	F	3	A

Dequeue A Update B & C

v	known	dv	pv
A	T	0	0
B	F	6	A
C	T	3	A

Dequeue C

Nothing to update because we dequeued A and C has no edges that it points to. The problem here is that there is shorter path to C going through B at a cost of 1, but it never get's updated since C doesn't know about B.

6. (7 pts): Weiss Exercise 9.38

*You are given a set of N sticks, which are lying on top of each other in some configuration. Each stick is specified by its two endpoints; each endpoint is an ordered triple giving its x , y , and z coordinates; no stick is vertical. A stick may be picked up only if there is no stick on top of it.**

- a. Explain how to write a routine that takes two sticks a and b and reports whether a is above, below, or unrelated to b . (This has nothing to do with graph theory.)*
- b. Give an algorithm that determines whether it is possible to pick up all the sticks, and if so, provides a sequence of stick pickups that accomplishes this.*

a. If we consider Z to be the vertical axis, I would suggest that the two sticks need to have Z in common in order to be considered as being above or below one another. So a routine could visit stick A and examine its adjacent vertices, keeping the value for Z in a list, if B is an adjacent vertex, examine its value for Z and compare it to B .previous' value for Z . If we find a match doing this, we can assume the A and B share edge Z , thus indicating the one is on-top or below the other.

b. Ran out of time to do this algorithm.